Créer un site vitrine avec Netvolution Studio BTS SIO 1re année — Bloc 1

Installer un CMS visuel, prendre en main l'éditeur, livrer un mini-site client

Web : CMS et site statique

Objectifs

- Installer un outil de développement web à partir de son dépôt Git et le lancer en local.
- Expliquer ce qu'est un CMS et ce que produit Netvolution Studio : un site **statique**.
- Construire un site de trois pages répondant au besoin d'un client et le vérifier sur mobile.
- Exporter le site et identifier les fichiers à déposer chez un hébergeur.

Prérequis

- Terminal : se déplacer dans un dossier, lancer une commande (SQ01).
- HTML : titres `h1` / `h2`, paragraphes, liens (SQ03 P01-P02).
- Client-serveur : un navigateur envoie une requête, un serveur répond (SQ03 P01).

Sommaire

Étape	Contenu

Mission et organisation

Votre agence web vient d'adopter **Netvolution Studio**, un CMS visuel : on compose les pages à la souris, comme dans un *page builder*, puis le Studio produit un site Astro **statique** (des fichiers HTML, CSS et images) qu'on peut héberger n'importe où. Aujourd'hui, vous l'installez puis vous livrez une première version du site d'un client.

En binôme sur un poste. L'un manipule, l'autre lit la fiche, vérifie et note ; changez de rôle à chaque étape. Chacun tient **ses propres notes** (réponses « À noter »), qui serviront à la restitution.

IA interdite : l'assistant IA du Studio n'est pas activé, et les textes du client sont fournis. Prendre en main un outil, respecter un besoin et vérifier son résultat sont les compétences travaillées.

Étape 1 — Installer et lancer le Studio

Vérifier les prérequis

Ouvrez un terminal (PowerShell sous Windows) et tapez :

```
node -v
npm -v
git --version
```

Il faut **Node.js 24 ou plus récent** (`v24.x` ou `v26.x`) et npm 11. Si `node` ou `git` est introuvable ou trop ancien, appelez l'enseignant : ne lancez pas d'installateur trouvé sur Internet.

Récupérer le code et ses dépendances

Placez-vous dans votre dossier de travail, puis :

```
git clone https://github.com/netvolutionfr/netvolution-studio.git
cd netvolution-studio
npm ci
```

- `git clone` copie le dépôt public du projet (le code source) sur le poste.

- `npm ci` installe **exactement** les bibliothèques listées dans `package-lock.json`, dans le dossier `node_modules`. Comptez de quelques secondes à quelques minutes selon le réseau.

Sans `git` ? Sur la page GitHub du projet : bouton **Code** › **Download ZIP**, puis extrayez l'archive et ouvrez le terminal dans le dossier extrait.

Lancer le Studio

```
npm run dev
```

Ouvrez **http://127.0.0.1:5173/** dans le navigateur. L'écran **Projets** s'affiche, en français si votre navigateur est en français. **Laissez le terminal ouvert** : le fermer, ou y taper Ctrl C, arrête le Studio.

À noter :

1. Taille du dossier `node_modules` (clic droit › Propriétés, ou `du -sh node_modules`). Pourquoi ce dossier ne se trouve-t-il pas dans le dépôt Git ?
2. Que signifie l'adresse `127.0.0.1` ? Votre voisin peut-il ouvrir **votre** Studio depuis son poste ?

Ça ne démarre pas ? Lisez le **dernier message** du terminal avant d'appeler : `EADDRINUSE` = le port 5173 est déjà pris (un autre Studio est lancé) ; `engine` ou `Unsupported` = version de Node trop ancienne ; `ENOTFOUND` ou `ETIMEDOUT` = réseau ou proxy.

Étape 2 — Premier projet : découvrir l'éditeur

Suivez le tutoriel officiel, accessible depuis le bouton **?** du Studio (chapitre « Prise en main en 20 minutes »), **étapes 1 à 7** :

1. Créez un projet nommé `essai-<vos initiales>` à partir du modèle **Atelier Nord — site vitrine (5 pages)**.
2. Repérez les cinq zones : pages et bibliothèque **à gauche**, **canvas** au centre, **inspecteur** à droite, barre d'outils en haut, arborescence en bas à gauche.
3. Modifiez le grand titre de l'accueil ; changez sa taille dans l'inspecteur.
4. Ajoutez une page « Tarifs » (adresse `tarifs`) avec la composition **Appel à l'action**, puis ajoutez-la au **menu principal**.
5. Passez en affichage **Mobile** : que devient le menu ? Revenez en **Desktop**.
6. Enregistrez (Ctrl S) : l'état passe à « Enregistré ».

À noter :

3. Sélectionnez le grand titre puis ouvrez l'arborescence : dans quel élément est-il rangé ? Quel **niveau** de titre (H1, H2…) l'inspecteur indique-t-il ?
4. Le projet est enregistré dans `data/projects/` (fichier `site.json`) : ouvrez ce dossier. Sous quelle forme le site est-il stocké ? Est-ce déjà du HTML ?

Réussite : la page Tarifs existe, apparaît dans le menu, et vous savez retrouver un élément dans le canvas, dans l'arborescence et dans l'inspecteur.

Étape 3 — Mini-projet : le site d'un client

Choisissez **un** client parmi les trois fiches ci-dessous (clients fictifs). Créez un **nouveau projet** à son nom à partir d'**Atelier Nord**, ou de **Cabinet Valéor** si vous voulez une charte plus sobre. Vous adaptez un modèle : on ne part pas d'une page blanche, mais tout ce qui reste à l'écran doit concerner le client.

Cahier des charges commun

Exigence	Vérification
Trois pages : **Accueil**, une page **Prestations** (ou Activités), **Contact**	Panneau Pages
Les autres pages du modèle sont supprimées, ou dépubliées	Œil barré ou page absente
Menu principal : les trois pages, dans cet ordre	Header du canvas
Nom du site, slogan et coordonnées du client dans le header et le footer	Header et footer
Un seul titre H1 par page, puis des H2	Inspecteur › Niveau
Aucun texte de démonstration (« Lorem ipsum », Atelier Nord…)	Relecture à deux
Une **meta description** par page (une phrase, 150 caractères maximum)	Inspecteur › Référencement
Lisible en **Mobile** : rien ne déborde, menu utilisable	Affichage Mobile
Chaque bouton mène à une page	Inspecteur › Lien

Images : gardez les médias de démonstration du modèle, ou n'en mettez pas. Ne copiez pas d'image trouvée sur Internet : elle a un auteur et des droits.

Client A — Le Fournil de Clara

Boulangerie-pâtisserie artisanale, 12 rue des Tilleuls, 02100 Saint-Quentin. Tél. 03 23 00 00 00 (numéro fictif). Ouvert du mardi au samedi, 6 h 30 – 19 h 30, dimanche 7 h – 12 h 30.

Slogan : « Du pain au levain, cuit chaque matin. »

Prestations : pains au levain et spéciaux ; viennoiseries pur beurre ; entremets et gâteaux sur commande (48 h à l'avance, par téléphone) ; plateaux pour les entreprises.

Besoin : les clients appellent sans cesse pour les horaires ; elle veut aussi faire connaître les commandes de gâteaux. ### Client B — Atelier Vélo Solidaire

Association loi 1901, 4 place de la Gare, 02100 Saint-Quentin. contact@avs-exemple.fr (adresse fictive). Atelier ouvert le mercredi 14 h – 18 h et le samedi 10 h – 17 h.

Slogan : « Réparez votre vélo, on vous montre comment. »

Activités : atelier d'autoréparation encadré par des bénévoles ; vente de vélos reconditionnés ; marquage antivol ; sorties découverte un dimanche par mois. Adhésion annuelle : 15 €, 5 € pour les étudiants.

Besoin : recruter des adhérents et des bénévoles, expliquer le fonctionnement de l'atelier.

Client C — Nova Informatique

Dépannage informatique pour particuliers et petites entreprises, 8 avenue Faidherbe, 02100 Saint-Quentin. Tél. 03 23 00 00 01 (numéro fictif). Du lundi au vendredi, 9 h – 18 h ; intervention à domicile sur rendez-vous.

Slogan : « Votre informatique remise sur pied, sans jargon. »

Prestations : diagnostic et réparation de PC ; installation d'une box et du Wi-Fi ; sauvegarde et récupération de données ; mise en place de la messagerie et des postes pour les TPE. Diagnostic en atelier : 30 €.

Besoin : rassurer des clients peu à l'aise avec l'informatique et donner envie d'appeler.

> **Méthode.** Commencez par le **menu et les pages** (squelette), puis le **texte de l'accueil**, puis les deux autres pages. Enregistrez souvent : le Studio n'enregistre pas automatiquement. Un élément introuvable dans le canvas ? Passez par l'**arborescence**. Une erreur ? Ctrl Z.

Étape 4 — Contrôler, exporter, expliquer

1. Sans rien sélectionner (Échap), ouvrez l'inspecteur du **site** : renseignez son nom, la langue `fr` et l'URL fictive `https://www.<client>.example`.

2. Menu ⋯ **Exporter le site (Astro)…**. Lisez les **contrôles avant export** : corrigez chaque **erreur bloquante** et chaque avertissement que vous comprenez (cliquer sur un message sélectionne l'élément concerné).
3. Cochez **Construire aussi le site statique**, puis **Exporter**. Patientez.
4. Téléchargez le **site statique** (`.tar.gz`), extrayez-le et ouvrez son contenu.
5. Menu ⋯ **Exporter l'archive du projet** : c'est votre sauvegarde. Déposez-la dans votre espace personnel.

À noter :

5. Listez les fichiers et dossiers du site statique. Où se trouve la page Contact ?
6. Ouvrez `index.html` dans VSCode : retrouvez votre `<h1>` et votre meta description.
7. Ce site a-t-il besoin du Studio, de Node.js ou d'une base de données pour fonctionner chez l'hébergeur ? Justifiez à partir de ce que vous voyez.

Réussite : le contrôle avant export ne signale plus d'erreur, le site statique est téléchargé, et vous savez expliquer ce qu'il faudrait déposer sur un serveur web.

Pour aller plus loin

- Ajoutez au client une page **Questions fréquentes** avec un **accordéon** (catégorie Structure), reliée au menu.
- Dans le **Design system**, changez la couleur principale pour l'adapter au client : observez que **toutes** les pages changent. Pourquoi est-ce préférable à la modification élément par élément ?
- Comparez à WordPress : où seraient stockés les textes, et qu'est-ce qui serait exposé en ligne ?

Clôture individuelle

Sans regarder les notes du binôme, complétez dans vos notes :

« Un CMS sert à… Netvolution Studio tourne sur… et n'est pas accessible depuis… parce que… Le site exporté est composé de… ; pour le mettre en ligne, il faudrait… »

- [ ] Le Studio est installé et je sais le relancer (`npm run dev` , puis l'adresse).
- [ ] Le site du client a trois pages, un menu, un H1 par page et aucun texte de démonstration.
- [ ] Le site est vérifié en mobile et le contrôle avant export ne signale aucune erreur.
- [ ] J'ai téléchargé le site statique et l'archive du projet.
- [ ] J'explique la différence entre l'outil (le Studio) et le produit (le site statique).

Corrigés

BTS SIO 1re année — Bloc 1

Installer un CMS visuel, prendre en main l'éditeur, livrer un mini-site client

Web : CMS et site statique

1. Plusieurs centaines de Mo (environ 440 Mo). `node_modules` se **reconstruit** à partir de `package.json` et `package-lock.json` avec `npm ci` : le versionner alourdirait le dépôt sans rien apporter (fichier `.gitignore`).

2. `127.0.0.1` est l'adresse de **boucle locale** : le poste lui-même. Le Studio n'écoute que sur cette adresse, donc le voisin ne peut pas l'ouvrir. C'est voulu : le Studio n'a **aucune authentification** ; ouvert au réseau, n'importe qui pourrait modifier ou supprimer les projets.

3. Le titre est rangé dans une section (le hero), elle-même dans la page ; c'est le **H1**, unique sur la page (référencement et accessibilité). On retrouve l'arbre du DOM vu en P03.

4. Le projet est un **document JSON** (avec ses médias) : une description structurée des pages, pas du HTML. Le HTML n'est produit qu'à l'**export** — le JSON est la source de vérité.

5. `index.html`, `contact/index.html` (chaque page a son dossier, comme dans Soft-Tech), `404.html`, `robots.txt`, `sitemap.xml` (l'URL du site est renseignée), dossier `_astro/` et images optimisées. La page Contact : `contact/index.html`, servie à l'adresse `/contact/`.

6. Le `<h1>` est dans le `<body>` ; la description est dans le `<head>` : `<meta name="description" content="…">`.

7. Non : ce sont des **fichiers statiques**. N'importe quel serveur web (Apache, nginx) les envoie tels quels ; il n'y a ni Node.js, ni PHP, ni base de données, ni espace d'administration en ligne — donc moins de choses à mettre à jour et à attaquer qu'un WordPress. Contrepartie : le client ne modifie pas son site lui-même en ligne ; toute modification passe par le Studio puis un nouvel export, et **tous** les fichiers en ligne sont remplacés.